

2-Player Reaction Game

Code Documentation

Arduino Mega Project

Hardware: Arduino Mega 2560 | 2× MAX7219 LED Matrix | 16×2 LCD

May 2026

Table of Contents

1. Project Overview
2. Game Flow & State Machine
3. Hardware & Pin Reference
4. Code Structure
5. Key Functions
6. The 90° Rotation Compensation
7. LED Pattern Reference
8. Edge Cases & Special Rules
9. Timing Reference

1. Project Overview

This is a two-player electronic reaction game built on an Arduino Mega 2560. Both players watch a pair of MAX7219 8×8 LED matrices count down from 3 to 0. After a random hidden delay, both matrices simultaneously display '0' as the GO signal. Each player slams their button as fast as possible. The player with the shorter reaction time wins — their matrix shows a smiley face while the loser's matrix shows a sad face. Reaction times are displayed on a 16×2 LCD.

Components Required

- Arduino Mega 2560
- 2× MAX7219 8×8 LED Matrix modules
- 1× 16×2 LCD (HD44780 compatible)
- 2× push buttons
- 2× 10 kΩ pull-down resistors (or use Arduino INPUT_PULLUP — no resistor needed)
- Jumper wires and breadboard
- LedControl library (install via Library Manager)
- LiquidCrystal library (built into Arduino IDE)

What Happens At a Glance

- **Power on:** LCD shows 'Press to start', matrices are off.
- **Start:** Either player presses their button to begin.
- **Countdown:** Both matrices show 3 → 2 → 1 at one-second intervals.
- **Random wait:** Matrices go blank for a random 1–4 second pause.
- **GO!:** Both matrices show 0. Players press as fast as possible.
- **Result:** Winner gets a smiley face, loser gets a sad face. LCD shows both times.
- **Replay:** Any button press returns to the IDLE screen.

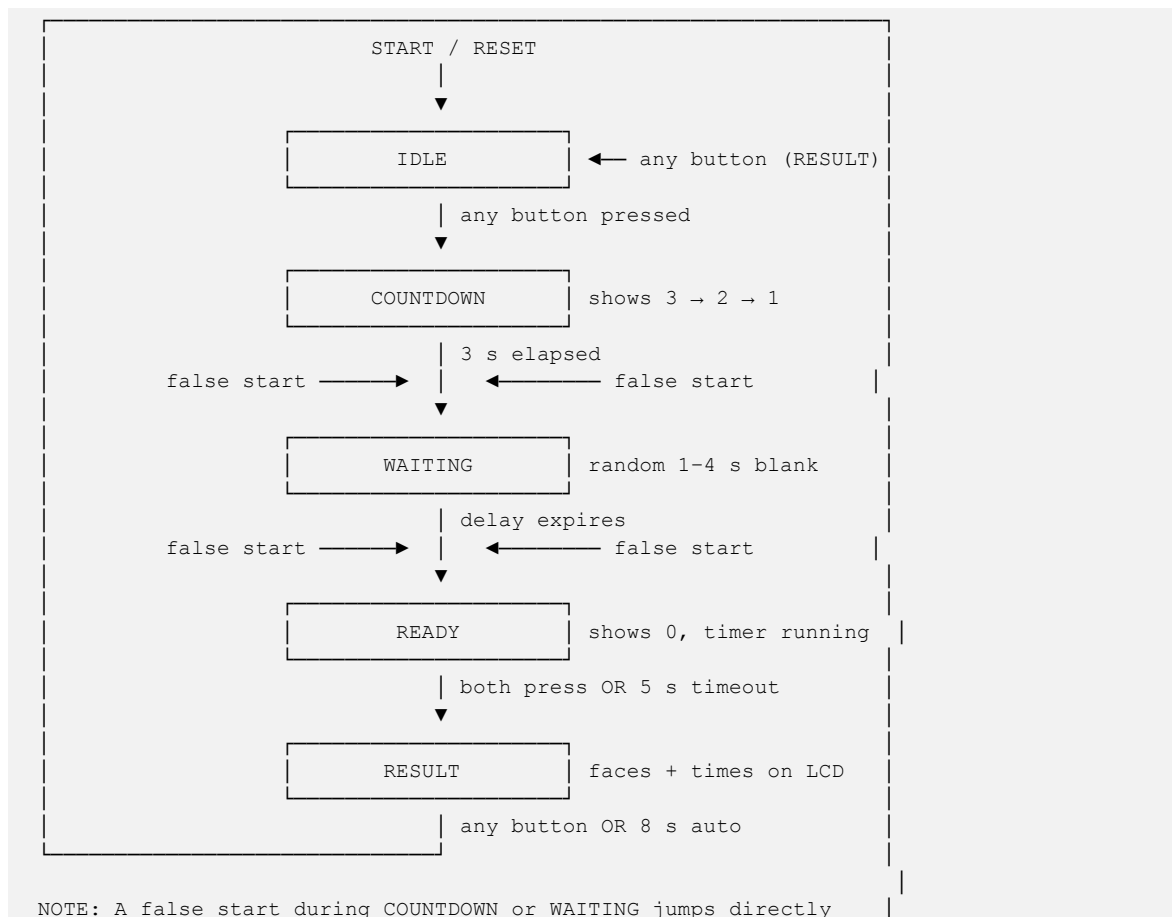
2. Game Flow & State Machine

The entire game is controlled by a finite state machine with five states. The current state is stored in the global variable `gameState` (of type `GameState` enum). All state transitions are handled by the `enterState()` function, which also updates the LCD and matrices for each new state.

States

State	Matrices Show	LCD Shows	Exits When
IDLE	Off (blank)	"2-Player Reflex! / Press to start"	Either button is pressed
COUNTDOWN	3 → 2 → 1 (1 s each)	"Get Ready!"	3 seconds elapsed
WAITING	Off (blank)	"Wait for it..."	Random delay (1–4 s) expires
READY	Both show 0 (GO!)	"GO!!! / Time: 00.00s" (live timer)	Both press OR 5 s timeout
RESULT	☺ winner / ☹ loser	Reaction times + winner announcement	Either button pressed OR 8 s auto

State Transition Diagram



to RESULT – the offending player loses automatically.

3. Hardware & Pin Reference

Left Matrix (Player 1 Side)

Matrix Pin	Arduino Mega Pin	Signal	Code Constant
DIN (Data)	11	Serial data out	LEFT_DIN = 11
CLK (Clock)	13	SPI clock	LEFT_CLK = 13
CS (Chip Select)	12	Active-low chip select	LEFT_CS = 12
VCC	5V	Power	—
GND	GND	Ground	—

Right Matrix (Player 2 Side)

Matrix Pin	Arduino Mega Pin	Signal	Code Constant
DIN (Data)	47	Serial data out	RIGHT_DIN = 47
CLK (Clock)	51	SPI clock	RIGHT_CLK = 51
CS (Chip Select)	49	Active-low chip select	RIGHT_CS = 49
VCC	5V	Power	—
GND	GND	Ground	—

16×2 LCD

LCD Pin	Arduino Mega Pin	Signal	Notes
RS	10	Register Select	0=command, 1=data
EN	9	Enable	Latches data on falling edge
D4	7	Data bit 4	4-bit mode; D0–D3 not connected
D5	6	Data bit 5	
D6	5	Data bit 6	
D7	4	Data bit 7	
V0	Pot wiper	Contrast	Use 10 kΩ potentiometer
VDD	5V	Power	
GND	GND	Ground	
A/LED+	5V via 220 Ω	Backlight +	
K/LED–	GND	Backlight –	

Buttons

Button	Arduino Mega Pin	Code Constant	Wiring
Player 1 (Left)	3	BTN_P1 = 3	Pin → button → GND (INPUT_PULLUP active)
Player 2 (Right)	43	BTN_P2 = 43	Pin → button → GND (INPUT_PULLUP

			active)
--	--	--	---------

*Both buttons use INPUT_PULLUP mode. The pin reads HIGH when the button is open and LOW when pressed.
No external pull-up resistor is required.*

4. Code Structure

The sketch is divided into seven logical sections, each serving a distinct purpose.

Libraries & Includes

```
#include <LiquidCrystal.h> and #include <LedControl.h>
```

Pull in the two third-party libraries. LedControl drives the MAX7219 chips over a software SPI bus. LiquidCrystal drives the HD44780-compatible LCD.

Hardware Objects

```
LedControl lcLeft / lcRight | LiquidCrystal lcd
```

Global objects that wrap the hardware. LedControl takes (DIN, CLK, CS, numDevices). LiquidCrystal takes (RS, EN, D4, D5, D6, D7). Declared globally so every function can access them.

LED Patterns

```
const byte DIGIT_0...SAD[8] and COUNTDOWN_PAT[]
```

Each pattern is an 8-element byte array. Each byte represents one row of the 8×8 matrix. bit7 is the leftmost LED, bit0 is the rightmost. Patterns are stored as they should visually appear — the rotation is applied at draw time by displayPattern().

Game Variables

```
gameState, stateStart, p1Time, p2Time, p1False, p2False, lastP1, lastP2...
```

All runtime state is held in global variables so any function can read or write them. p1Time and p2Time are long (signed) to allow -1 as a sentinel meaning 'not yet pressed'.

Timing Constants

```
CD_STEP_MS, TIMEOUT_MS, RESULT_MS
```

Three unsigned long constants control how long each phase lasts. They are defined at the top for easy tuning without hunting through the code.

Helper Functions

```
displayPattern, fillAll, initMatrix, startupFlash, lcdLine, resultLine
```

Utility functions that perform one specific hardware task. Keeping them separate makes the main game logic shorter and easier to read.

Game Logic

```
showResult, enterState, readButtons, setup, loop
```

The core state machine. enterState() drives transitions. readButtons() detects presses and updates state. loop() handles timed transitions.

5. Key Functions

displayPattern(lc, pat)

Sends an 8-byte visual pattern to a MAX7219, applying a 90° counter-clockwise rotation to compensate for the physical 90° clockwise mount of the module. See Section 6 for the full mathematical derivation.

```
void displayPattern(LedControl &lc, const byte pat[8]) {
    for (int p = 0; p < 8; p++) {
        byte row = 0;
        for (int q = 0; q < 8; q++)
            if ((pat[q] >> p) & 1) row |= (1 << (7 - q));
        lc.setRow(0, p, row);
    }
}
```

The outer loop iterates over physical rows ($p = 0..7$). The inner loop iterates over all 8 source (visual) rows (q) and picks out bit p from each, assembling them into one physical row byte. The resulting byte is sent with `lc.setRow(0, p, row)`.

fillAll(lc, val)

Sets every row of a matrix to the same byte value. Called with 0xFF to turn all 64 LEDs on (startup flash) or 0x00 to clear the display.

```
void fillAll(LedControl &lc, byte val) {
    for (int r=0; r<8; r++) lc.setRow(0, r, val);
}
```

initMatrix(lc)

Wakes the MAX7219 from power-down mode (`shutdown(false)`), sets brightness to level 8 out of 15, and clears all LEDs. Called once per matrix in `setup()`.

```
void initMatrix(LedControl &lc) {
    lc.shutdown(0, false);
    lc.setIntensity(0, 8);
    lc.clearDisplay(0);
}
```

startupFlash(lc)

Plays a two-flash animation on power-up: all LEDs on (200 ms) → off (100 ms) → on (200 ms) → off (200 ms). This gives the user immediate visual confirmation that the matrix is wired correctly before the game screen appears.

```
void startupFlash(LedControl &lc) {
    fillAll(lc, 0xFF); delay(200);
    fillAll(lc, 0x00); delay(100);
    fillAll(lc, 0xFF); delay(200);
}
```

```
fillAll(lc, 0x00); delay(200);
}
```

lcdLine(row, s)

Writes a string to a specific LCD row (0 or 1), then pads with spaces up to 16 characters. The padding is essential — without it, leftover characters from a longer previous message would remain visible on the display.

```
void lcdLine(uint8_t row, const char *s) {
    lcd.setCursor(0, row);
    int i = 0;
    while (s[i] && i < 16) lcd.print(s[i++]);
    while (i++ < 16)        lcd.print(' '); // ← padding clears old text
}
```

resultLine(row, prefix, ms, isFalse, wins)

Formats and writes one 16-character result line to the LCD. The prefix is always 'P1:' or 'P2:' (3 characters), leaving 13 characters for the result. Four formats are possible:

Condition	LCD Output (16 chars)	Example
False start	PX:FALSE START	P1:FALSE START
Timed out	PX: TIMEOUT	P2: TIMEOUT
Winner	PX: XXXms WINS!	P1: 234ms WINS!
Loser	PX: XXXms	P2: 567ms

snprintf is used to build the 16-char string into a local char `buff[17]` buffer before calling `lcdLine()` to write it.

showResult()

Evaluates all possible game outcomes and updates both matrices and the LCD accordingly. It runs in two passes:

- **Pass 1 — Logic:** Determine winner: a chain of if/else checks sets bool flags `p1w` and `p2w`.
- **Pass 2 — Display:** Act on the result: matrices get SMILEY or SAD, and `resultLine()` formats the LCD.

The full decision tree:

Situation	p1w	p2w	Matrices
Both false-started	✗	✗	Both blank
P1 false-started only	✗	✓	P1=sad, P2=smiley
P2 false-started only	✓	✗	P1=smiley, P2=sad
Both timed out	✗	✗	Both blank

P1 timed out (P2 reacted)	✗	✓	P1=sad, P2=smiley
P2 timed out (P1 reacted)	✓	✗	P1=smiley, P2=sad
P1 faster reaction	✓	✗	P1=smiley, P2=sad
P2 faster reaction	✗	✓	P1=sad, P2=smiley
Exact tie (same ms)	✓	✓	Both smiley

enterState(GameState s)

Transitions the game to a new state. It always saves the current timestamp (`stateStart = millis()`) and then performs the entry action for the new state: clearing or drawing on the matrices and writing the appropriate LCD message. For WAITING it also generates the random delay: `waitDur = random(1000, 4001)`. For RESULT it calls `showResult()` to calculate and display the outcome.

readButtons()

Called every loop iteration. Reads both button pins and detects falling edges (HIGH → LOW transition with INPUT_PULLUP). Edge detection prevents a held button from registering multiple presses. The detected press is then acted on depending on the current state:

State	P1 / P2 pressed	Action
IDLE	Either	enterState(COUNTDOWN)
COUNTDOWN or WAITING	Either	Mark as false-start → enterState(RESULT)
READY	P1	Record <code>p1Time = millis() - stateStart</code>
READY	P2	Record <code>p2Time = millis() - stateStart</code>
READY (both done)	Both	enterState(RESULT) immediately
RESULT	Either	enterState(IDLE) — restart game

Note: `p1Time` and `p2Time` are set to -1 at the start of each round. The guard (`p1Time < 0`) ensures a player's time is only recorded once even if they mash the button repeatedly.

6. The 90° Rotation Compensation

Why Is Rotation Needed?

The MAX7219 LED matrix modules are mounted rotated 90° clockwise in this project (the connector points sideways). If patterns were sent without correction, every digit and symbol would appear rotated 90° clockwise on screen. To compensate, a 90° counter-clockwise rotation is applied in software inside `displayPattern()`.

Physical vs. Visual Mapping

Normal orientation (how LedControl sees it)	After 90° CW physical rotation (how we see the module on the board)
Physical col: 0 1 2 3 4 5 6 7 row 0: . . X X X X . . row 1: . X X (If no correction applied, the pattern would appear rotated clockwise)	Visual appearance col: 0 1 2 3 4 5 6 7 row 0: . . X X X X . . ← physical COL 0 (bit7) row 1: X X X X ← physical COL 1 ... (If no correction applied, the pattern would appear rotated clockwise)

The Mathematical Derivation

When a matrix is rotated 90° clockwise physically, the coordinate mapping is:

<code>visual(row=vr, col=vc) → physical(row = 7-vc, col = vr)</code>
In other words: - Physical row p corresponds to visual column c = 7-p - Physical col q corresponds to visual row r = q
To build physical row p, we collect one bit from each visual row q: <code>physical_byte[p] += bit(p) of visual_byte[q] placed at bit position (7-q)</code>
In code: <pre>for q in 0..7: if (pat[q] >> p) & 1: // extract bit p from visual row q row = (1 << (7 - q)) // place it at bit (7-q) of physical row p</pre>

This transform is applied once per pattern, once per matrix update. It runs in 64 iterations (8×8) and takes negligible time on the Arduino Mega.

Verification Example

Consider the top-left visual pixel (vr=0, vc=0). Applying the formula: physical row = 7-0 = 7, physical col = 0 (bit 7 of `physical_byte[7]`). When the matrix is viewed after 90° CW rotation, physical row 7 (bottom) becomes the visual left side, and physical col 0 (leftmost bit) becomes the visual top. So `physical(7, 0)` appears at the visual top-left — correct!

7. LED Pattern Reference

Each pattern is stored as 8 bytes. In the ASCII grids below, 'X' = LED on, '.' = LED off. Column 0 (leftmost) corresponds to bit 7 (MSB) of each byte. These are the visual patterns — the rotation shown in Section 6 is applied at draw time.

Digit 0

```
const byte DIGIT_0[8] = { 0x3C, 0x42, 0x42, 0x42, 0x42, 0x42, 0x42, 0x3C };

Col: 0 1 2 3 4 5 6 7      Hex
Row 0: . . X X X X . .    0x3C
Row 1: . X . . . . X .    0x42
Row 2: . X . . . . X .    0x42
Row 3: . X . . . . X .    0x42
Row 4: . X . . . . X .    0x42
Row 5: . X . . . . X .    0x42
Row 6: . X . . . . X .    0x42
Row 7: . . X X X X . .    0x3C
```

Digit 1

```
const byte DIGIT_1[8] = { 0x10, 0x30, 0x10, 0x10, 0x10, 0x10, 0x10, 0x38 };

Col: 0 1 2 3 4 5 6 7      Hex
Row 0: . . . X . . . .    0x10
Row 1: . . X X . . . .    0x30
Row 2: . . . X . . . .    0x10
Row 3: . . . X . . . .    0x10
Row 4: . . . X . . . .    0x10
Row 5: . . . X . . . .    0x10
Row 6: . . . X . . . .    0x10
Row 7: . . X X X . . .    0x38
```

Digit 2

```
const byte DIGIT_2[8] = { 0x3C, 0x42, 0x02, 0x04, 0x08, 0x10, 0x20, 0x7E };

Col: 0 1 2 3 4 5 6 7      Hex
Row 0: . . X X X X . .    0x3C
Row 1: . X . . . . X .    0x42
Row 2: . . . . . . X .    0x02
Row 3: . . . . . X . .    0x04
Row 4: . . . . X . . .    0x08
Row 5: . . . X . . . .    0x10
Row 6: . . X . . . . .    0x20
Row 7: . X X X X X X .    0x7E
```

Digit 3

```
const byte DIGIT_3[8] = { 0x3C, 0x42, 0x02, 0x3E, 0x02, 0x02, 0x42, 0x3C };
```

Col:	0	1	2	3	4	5	6	7	Hex
Row 0:	.	.	X	X	X	X	.	.	0x3C
Row 1:	.	X	.	.	.	.	X	.	0x42
Row 2:	.	.	.	.	.	.	X	.	0x02
Row 3:	.	.	X	X	X	X	X	.	0x3E
Row 4:	.	.	.	.	.	.	X	.	0x02
Row 5:	.	.	.	.	.	.	X	.	0x02
Row 6:	.	X	.	.	.	.	X	.	0x42
Row 7:	.	.	X	X	X	X	.	.	0x3C

Smiley face

```
const byte SMILEY[8] = { 0x3C, 0x42, 0xA5, 0x81, 0xA5, 0x99, 0x42, 0x3C };
```

Col:	0	1	2	3	4	5	6	7	Hex
Row 0:	.	.	X	X	X	X	.	.	0x3C
Row 1:	.	X	.	.	.	.	X	.	0x42
Row 2:	X	.	X	.	.	X	.	X	0xA5
Row 3:	X	.	.	.	.	.	.	X	0x81
Row 4:	X	.	X	.	.	X	.	X	0xA5
Row 5:	X	.	.	X	X	.	.	X	0x99
Row 6:	.	X	.	.	.	.	X	.	0x42
Row 7:	.	.	X	X	X	X	.	.	0x3C

Sad face

```
const byte SAD[8] = { 0x3C, 0x42, 0xA5, 0x81, 0x99, 0xA5, 0x42, 0x3C };
```

Col:	0	1	2	3	4	5	6	7	Hex
Row 0:	.	.	X	X	X	X	.	.	0x3C
Row 1:	.	X	.	.	.	.	X	.	0x42
Row 2:	X	.	X	.	.	X	.	X	0xA5
Row 3:	X	.	.	.	.	.	.	X	0x81
Row 4:	X	.	.	X	X	.	.	X	0x99
Row 5:	X	.	X	.	.	X	.	X	0xA5
Row 6:	.	X	.	.	.	.	X	.	0x42
Row 7:	.	.	X	X	X	X	.	.	0x3C

8. Edge Cases & Special Rules

False Start (one player)

If one player presses their button during COUNTDOWN or WAITING, they are immediately flagged as a false starter (`p1False` or `p2False` = true) and the game jumps straight to RESULT. The false starter receives a sad face regardless of any time they might have recorded. The other player wins by default — their matrix shows a smiley face and the LCD shows their time as '----' (they hadn't pressed yet).

False Start (both players simultaneously)

If both players press before GO!, both false flags are set and both matrices go blank. The LCD announces no winner. This is handled by the same code path — both `p1w` and `p2w` remain false in `showResult()`.

Timeout — one player doesn't press

If READY lasts longer than `TIMEOUT_MS` (5 seconds) without both players pressing, `enterState(RESULT)` is called automatically. Any player who didn't press has a time of -1 (never set), which `showResult()` treats as a timeout — that player receives a sad face and the LCD shows 'TIMEOUT'. The other player wins.

Timeout — neither player presses

If nobody presses within 5 seconds, both `p1Time` and `p2Time` remain -1. `showResult()` detects this (neither `p1ok` nor `p2ok`), sets no winner, blanks both matrices, and shows 'Nobody reacted!'

Exact Tie

If both players press in the same millisecond (`p1Time == p2Time`), the code sets both `p1w` and `p2w` to true. Both matrices display a smiley face and the LCD shows both times followed by 'TIE! Both win!'

Auto-return to IDLE

After `RESULT_MS` (8 seconds), the game automatically calls `enterState(IDLE)` without any button press. This prevents the result screen from staying on forever if neither player touches the board.

Non-blocking design

The main `loop()` never calls `delay()` during gameplay. All timing is done by comparing `millis()` - `stateStart` against a threshold. This means the button scanner (`readButtons`) and LCD timer updater run every loop iteration (~1 ms), giving accurate reaction time measurements and responsive button detection.

Random seed

`randomSeed(analogRead(A0))` is called in `setup()`. Analogue pin A0 is left floating, so it reads electrical noise — a different value each power-on. This ensures the random wait delay is unpredictable every game.

9. Timing Reference

All timing constants are defined as unsigned long at the top of the sketch. Changing them requires no modification to the game logic — only the constants need updating.

Constant	Value	Phase It Controls	What Changing It Does
CD_STEP_MS	1000 ms (1 s)	COUNTDOWN	How long each digit (3, 2, 1) is shown. Increase for slower countdown.
TIMEOUT_MS	5000 ms (5 s)	READY	How long players have to press after GO! Increase for more relaxed play.
RESULT_MS	8000 ms (8 s)	RESULT	How long the result screen is held before auto-returning to IDLE.
waitDur (runtime)	random 1000-4001 ms	WAITING	Regenerated each round. Change random(1000,4001) to adjust min/max wait.

Full Round Timeline (typical)

```
t = 0 ms    IDLE → COUNTDOWN    (player presses start)
t = 0 ms    Matrix shows "3"
t = 1000 ms Matrix shows "2"
t = 2000 ms Matrix shows "1"
t = 3000 ms COUNTDOWN → WAITING
t = 3000-   Matrices blank (random 1000-4000 ms pause)
t = 4000-7000 ms WAITING → READY (GO! signal - "0" shown)
t = GO + X ms    P1 presses → p1Time = X
t = GO + Y ms    P2 presses → p2Time = Y
t = GO + max(X,Y) READY → RESULT  (or t = GO + 5000 if timeout)
t = RESULT + 8 s RESULT → IDLE    (auto-return)
```

LCD Timeline During READY

While the game is in the READY state, the LCD row 1 is updated every 50 ms with a running clock formatted as 'Time: 00.00s'. The update is non-blocking — it only writes when (millis() - lastLcdRefresh) >= 50. This provides players with live feedback on how long the round has been active.